# Dockerfile Flakiness: Characterization and Repair

Taha Shabani
*University of British Columbia*
Vancouver, Canada
taha.shabani@ece.ubc.ca

Noor Nashid
*University of British Columbia*
Vancouver, Canada
nashid@ece.ubc.ca

Parsa Alian
*University of British Columbia*
Vancouver, Canada
palian@ece.ubc.ca

Ali Mesbah
*University of British Columbia*
Vancouver, Canada
amesbah@ece.ubc.ca

2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE) | 979-8-3315-0569-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSE55347.2025.00238

***Abstract*—Dockerfile flakiness—unpredictable temporal build failures caused by external dependencies and evolving environments—undermines deployment reliability and increases debugging overhead. Unlike traditional Dockerfile issues, flakiness occurs without modifications to the Dockerfile itself, complicating its resolution. In this work, we present the first comprehensive study of Dockerfile flakiness, featuring a nine-month analysis of 8,132 Dockerized projects, revealing that around 10% exhibit flaky behavior. We propose a taxonomy categorizing common flakiness causes, including dependency errors and server connectivity issues. Existing tools fail to effectively address these challenges due to their reliance on pre-defined rules and limited generalizability. To overcome these limitations, we introduce FLAKIDOCK, a novel repair framework combining static and dynamic analysis, similarity retrieval, and an iterative feedback loop powered by Large Language Models (LLMs). Our evaluation demonstrates that FLAKIDOCK achieves a repair accuracy of 73.55%, significantly surpassing state-of-the-art tools and baselines.**

***Index Terms*—Docker, Flakiness, Large Language Models, Automated Program Repair**

## I. INTRODUCTION

Docker [1] is a set of platform-as-a-service (PaaS) products that use OS-level virtualization to automate building, deploying, and delivering applications in containers. This approach simplifies deployment across diverse systems and plays a critical role in the CI/CD pipeline, highlighting the importance of ensuring Dockerfile reliability during builds to maintain software dependability.

Existing academic [2]–[9] and industrial efforts mainly focus on identifying Dockerfile smell or bug patterns. Static analysis studies [3], [8], [10], [11] offer a linter to detect smells inside Dockerfiles. While existing techniques effectively identify various Dockerfile smells, they often rely on pre-defined rules that are frequently outdated or poorly maintained [11]. PARFUM [5] enriches the Dockerfile AST with structural information derived from command lines, enabling the automatic detection and repair of smells. SHIPWRIGHT [7] employs a human-in-the-loop approach to repair broken Dockerfiles by clustering failure patterns using a modified BERT model [12] and HDBSCAN [13], then formalizing solutions into a repair pattern database with regular expressions and fix functions.

Unlike test flakiness [14], Dockerfile builds can exhibit *temporal failures* that emerge over time due to dynamic changes in external dependencies, such as updates to base images, third-party libraries, or environmental settings—occurring without any modifications to the Dockerfile itself. These temporal failures can manifest as either deterministic or non-deterministic behaviours, depending on the nature of the underlying issues. We refer to this phenomenon as *Dockerfile flakiness*—the unpredictable temporal behaviour of the Dockerfile build process, where builds may fail over time even though the content of the Dockerfile itself remains unchanged. Deterministic failures consistently cause builds to fail after a specific point, often due to changes such as the deprecation of a base image or the removal of a required dependency. In contrast, non-deterministic failures occur sporadically, resulting in random build successes and failures. Such issues are typically caused by transient factors, including server connectivity problems or temporary inconsistencies in package manager caches.

Existing studies [2]–[8], [10], [11] assume the reliability of the Dockerfiles and their build output over time, so that a build failure indicates the changes applied directly to the source Dockerfile. The literature largely overlooks the issue of flakiness in Dockerfile builds, which can disrupt the deployment process, particularly in CI/CD pipelines, by hindering automatic builds. This leads to delays and consumes valuable developer time and effort in diagnosing and resolving the underlying causes of the flakiness.

In this work, we first examine the consistency and reliability of Dockerfile builds over time. In a longitudinal study spanning a nine-month timeframe, we built and analyzed Dockerfiles of 8,132 open-source projects and observed that 798 (9.81%) exhibit flakiness behavior. Based on the observed instances of flakiness, we developed a taxonomy for characterizing Dockerfile flakiness. Our findings indicate that the most common types of flakiness are related to dependencies, web server connectivity, and security/authentication issues.

Second, we observe that existing techniques, such as PARFUM and SHIPWRIGHT, exhibit significant limitations in repairing flaky Dockerfiles. PARFUM primarily addresses code smells, making it less effective in tackling flakiness issues.

1558-1225/25/$31.00 ©2025 IEEE
DOI 10.1109/ICSE55347.2025.00238



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

SHIPWRIGHT, on the other hand, faces challenges including: (1) heavy reliance on human intervention, which hinders scalability and introduces variability due to dependence on the supervisor's expertise, (2) limited generalizability of repair patterns to diverse or unseen errors, (3) reliance on pattern-matching and regular expressions, which can introduce inconsistencies, and (4) the need for ongoing maintenance of the repair database to accommodate emerging failure types.

To overcome these shortcomings, we present FLAKIDOCK, a novel approach that leverages feedback-directed retrieval-augmented generation (RAG) with large language models (LLMs) to automatically repair Dockerfile flakiness. By integrating static and dynamic analysis, similarity retrieval, and an iterative LLM-based feedback loop, FLAKIDOCK effectively resolves complex flakiness issues.

In this work, we make the following contributions:

- The first study of Dockerfile flakiness characterization. We present the first taxonomy of Dockerfile flakiness by building Dockefiles and analyzing build outputs of 8,132 Dockerized projects.
- A dataset called FLAKE4DOCK including 798 and 7,325 flaky and non-flaky Dockerfiles. Flaky Dockerfiles are accompanied by categorization and build errors. We also provide repair information for 100 flaky Dockerfiles, specifically designed to evaluate flakiness detection and repair tasks.
- An iterative LLM-based technique called FLAKIDOCK that leverages both static and dynamic information from Dockerfiles to repair flakiness.
- An empirical evaluation of FLAKIDOCK, assessing its effectiveness with Dockerfile repair tools and baselines such as GPT-4.

Our results show that FLAKIDOCK achieves a 73.55% repair accuracy in resolving Dockerfile flakiness, significantly outperforming PARFUM (0.58%), SHIPWRIGHT (5.52%), and GPT-4-based prompting with build output (37.79%).

## II. MOTIVATION

Listing 1 shows a flaky Dockerfile, the corresponding build failure, and a subsequent repair based on the build error. According to the build output (lines 95 to 114), using a virtual environment is required to install `pip` for the specified `alpine` base image in Dockerfile (line 1) due to the adaptation of `Python Enhancement Proposal (PEP) 668` [15], which addresses Externally Managed Environments. This specification prevents package managers such as `pip` from modifying packages in the interpreter's default environment, ensuring compatibility and reducing the risk of breaking the underlying operating system managed by external package managers. As such, this Dockerfile fails to build, whereas previous builds prior to this adaptation were all successful. The solution here would be to create and activate a virtual environment, as shown in the Dockerfile context (lines 11 and 12).

While adhering to best practices [3], [5], [8], [10], [11] is essential to mitigate errors and vulnerabilities in Dockerfiles and in some cases to prevent potential failures from happening, we argue that this alone is insufficient to address flakiness. For example, existing tools such as HadoLint [10] recommend pinning the exact version of the base image (e.g., rule: DL3006) or dependencies (e.g., rules: DL3007, DL3008, DL3013, DL3016, DL3018) to prevent errors caused by their internal changes. This practice can be applied to Listing 1 by using old or outdated `alpine` images as a solution. However, it does not provide a viable solution for the problem; applying such rules without considering the static and dynamic nature of Dockerfiles can introduce other types of flakiness, such as outdatedness and compatibility issues in the future.

**Build Output**

```
1   ...
87  > [5/5] RUN pip3 install -r requirements.txt:
88  error: externally-managed-environment
95      If the package in question is not packaged already (
            and
96      hence installable via "apk add py3-somepackage"),
            please
97      consider installing it inside a virtual environment,
             e.g.:
98      ...
114 hint: See PEP 668 for the detailed specification.
115 ERROR: process "/bin/sh -c pip3 install -r requirements.
        txt" did not complete successfully: exit code: 1
```

**Repaired Dockerfile**

```
1   FROM alpine:latest
2   RUN apk add --update python3 py3-pip git tcpdump
3   RUN git clone https://github.com/649/Memcrashed-DDoS-
        Exploit.git Memcrashed
4   WORKDIR Memcrashed
5   ...
10  - RUN pip3 install -r requirements.txt
11  + RUN python3 -m venv venv
12  + RUN . venv/bin/activate && pip install -r requirements
        .txt
13  ENTRYPOINT ["python3", "Memcrashed.py"]
```

Listing 1: Base Image Internal Change

Listing 2 demonstrates another flaky Dockerfile that clings to the version pinning rule for the base image. As depicted in the Dockerfile (line 1), although the base image version is explicitly mentioned, inconsistent behavior is plausible due to using a relatively old base image. This flakiness is evident inside the build output (line 132), where the expression `pre_go17` is located. The error stems from the compatibility issue of a stale `GOLANG` base image, i.e., older than 1.17, with existing dependencies utilized in the Dockerfile (line 8), failing the compilation and build of the project. Accordingly, a base image version upgrade is required (line 2). Furthermore, updated `GOLANG` images require a different approach for handling executables (lines 3 and 4) due to the adoption of new techniques.

**Build Output**

```
1   ...
130 > [build-env 4/4] RUN cd /src && go build -ldflags "-
        linkmode external -extldflags -static" -o proxy:
```



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

```
/go/src/golang.org/x/net/context/pre_go17.go:47:2:
     background redeclared in this block
...
ERROR: process "/bin/sh -c cd /src && go build -ldflags
     \"-linkmode external -extldflags -static\" -o proxy
     " did not complete successfully: exit code: 2
```

**Repaired Dockerfile**

```
- FROM golang:1.9.1 AS build-env
+ FROM golang:1.22 AS build-env
+ WORKDIR /src
+ RUN go mod init my_module
RUN go get -d -v github.com/armon/go-socks5
...
RUN cd /src && go build -ldflags "-linkmode external -
     extldflags -static" -o proxy
# final stage
FROM scratch
WORKDIR /app
...
CMD ["./proxy"]
```

Listing 2: Compatibility Issues With Stale Base Image

Dockerfiles depend on various elements such as operating systems, packages, environments, commands, and project source code, making them susceptible to diverse forms of flakiness. Listings 1 and 2 demonstrate that understanding and resolving such flaky behavior requires analyzing both the static context (Dockerfile) and dynamic context (build output) along with its temporal changes.

## III. Characterization of Dockerfile Flakiness

In this section we present our longitudinal investigation of Dockerfile builds, which addresses the following research questions:

- **RQ1**: How prevalent is flakiness in Dockerfiles?
- **RQ2**: What are the main categories of Dockerfile flakiness?

The entire analysis, including project checkouts and Dockerfile builds, was conducted on an infrastructure comprising four Intel Xeon 2.50GHz machines, each with 62 GB of RAM.

### A. Data Collection

For our study, we started with the Shipwright dataset [7], which contains 20,526 Docker projects with ten or more stars from GitHub repositories. The dataset includes Docker projects created up to June 2020 and focuses exclusively on projects with a single Dockerfile located in the root directory. We cloned the most up-to-date version of all the repositories from the Shipwright dataset. However, some repositories were no longer publicly accessible, had been removed from GitHub, or no longer contained the root Dockerfiles. Consequently, our initial pool of projects comprised 18,055 repositories.

In our study, we analyze Dockerfiles along with their build outputs, which we generate by building the Dockerfiles within our infrastructure. Given that some Docker projects can be time-consuming to build, we set a 30-minute build timeout for each repository. This resulted in 93% of builds completing without a timeout. To ensure the reliability and efficiency of our large-scale Docker build system, we use the `docker build` command with the `--no-cache` option to eliminate failures stemming from cached Docker data. Additionally, we develop a systematic Docker cleaning technique to prevent environmental and internal errors, ensuring a fresh Docker environment. To enhance time efficiency and prevent internal errors, we clean the Docker system after every four consecutive builds—a frequency determined through trial and error. The cleaning process involves removing all the cache, images, and any other peripheral leftover data during the builds alongside uninstalling and reinstalling Docker with a steady version to prevent working with a corrupted Docker system. This pipeline ensures the freshness of the Docker system throughout the builds. Using this approach, we stored 32 rounds of builds for the initial Docker projects, capturing Dockerfile build outputs in our infrastructure machines from April 2023 to December 2023.

### B. Flakiness Extraction

The detection of flakiness within test suites has been extensively explored in the prior work [16], [17]. These efforts revolve around static and dynamic test code analysis, pattern recognition, rerunning tests multiple times, and checking code changes through times in repositories. Considering the complexity and variety of Dockerfile commands, external dependencies, and configurations, directly checking the Dockerfile context for flakiness detection would require an extensive endeavor. Furthermore, due to the dynamic nature of Dockerfiles, characterizing flakiness utilizing only the static analysis approaches would fail to address all aspects.

**Filtering Phase.** To start our flakiness study, we initiated an *in-context* Docker build for all the 18,055 Dockerized GitHub projects in our dataset that completed the build within 30-minutes. This resulted in 8,132 (45.04%) successful and 9,923 (54.96%) failed Docker builds. We consider only the successful ones as valid candidates for our flakiness analysis due to their initial stability.

Over a period of nine months, we rebuilt the 8,132 candidate projects weekly totaling 32 times. The reason for conducting builds over an extended timeframe was two-fold. First, building Dockerfiles is a time-consuming operation. The average time to build a single Dockerfile in our dataset is around eight minutes. Despite distributing our candidates across four different machines with similar operating systems and configurations and employing multiprocessing on each machine to enhance build speed, some degree of delay was unavoidable. Second, we expected that such an extended timespan could allow us to investigate temporal fluctuations in project stability.

**Pre-Processsing Phase.** Dockerfile build outputs can range from a few lines to thousands, detailing the progress of each execution step. Diagnosing and extracting critical failure points from these outputs is essential but challenging, especially when error logs are lengthy or the failure point is distant from



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

the error's manifestation. For instance, the `RUN pip install -r requirements.txt` command can produce hundreds of lines of output, often cluttered with warnings or unrelated messages, obscuring the root cause. To address this, we implemented a rule-based pre-processing approach that divides build outputs into stages, each corresponding to a specific Dockerfile command. Each stage includes lines containing the stage number, execution time, and log information. By analyzing each stage, we capture lines with error-related expressions like `ParseError` or `ERR!`, along with adjacent lines sharing the same execution time, to provide additional context for diagnosing failures.

### C. Flakiness Categorization

A taxonomy of Dockerfile flakiness is critical for systematically addressing inconsistencies and complexities, enabling developers to identify and resolve related challenges. We employ similarity analysis to remove duplicates or identical errors within Dockerfile build outputs. This is followed by a detailed examination of pre-processed outputs, leveraging GPT-3.5 to interpret and summarize the results. Finally, manual analysis is performed to address potential inaccuracies.

**Clustering Phase.** During the rebuild period, we encountered 12,964 failing build outputs while building the candidate projects. To streamline our analysis and minimize redundancy within Dockerfile build outputs, we conduct a clustering process using sentence similarity assessment within each project. To identify unique build errors in a single project, we measure the cosine similarity between the build output embeddings, extracted from the `all-mpnet-base-v2` [18] sentence transformer, which is trained over 1 billion sentences in different domains. Using the similarity scores, we cluster them into distinct groups. In our clustering approach, each new build output is evaluated for its similarity to existing clusters. Based on the average similarity with all current clusters, we decide whether to incorporate it into an existing cluster or create a new one. We apply this clustering method within individual projects rather than across all projects due to the intricate nature of Dockerfiles and their build outputs, which complicates the accurate clustering of errors when applied globally. To improve accuracy, we use pre-processed build outputs for similarity comparison rather than raw outputs. This approach resulted in an 87% reduction in failing build outputs, leaving 1,684 remaining, simplifying subsequent analysis steps.

**Labeling Phase.** We employed GPT-3.5 to extract error descriptions encountered during builds. Given the Dockerfile context alongside the corresponding build error, the model is prompted to extract a list of contributing factors to the error and an initial label indicating the category of error. We then used these build errors, alongside the information generated by the language model, to construct a taxonomy of Dockerfile build flakiness. Our objective was to create a comprehensive hierarchical classification that captures the diverse and dynamic behavior of Dockerfiles. To achieve this, a brainstorming session among authors was conducted to design a pipeline for analyzing context and refining the labels suggested by the language model. Two authors reviewed and resolved discrepancies in the labels generated by the model, based on the Dockerfiles and build errors. Any differences in interpretation were discussed among the authors to reach a consensus. This systematic analysis resulted in the generation of category hierarchies, requiring approximately 220 person-hours of effort.

### D. RQ1: Prevalence of Dockerfile Flakiness

While categorizing, we selectively omitted failures stemming from our infrastructure, Docker servers, and project source code issues to hone in on genuine instances of Dockerfile flakiness. Infrastructure failures accounted for 38 of the 1,684 builds, highlighting the effectiveness of our systematic Docker cleaning method in ensuring reliability. Issues stemming from Docker servers and project-specific errors accounted for 431 and 241 failures, respectively, and were also excluded from the flaky build outputs.

For instance, a project-specific error could involve a command such as `RUN build_script.sh`, which executes a script exhibiting non-deterministic behavior due to race conditions or reliance on external services. Similarly, `RUN npm install` can result in project-specific errors when dependencies listed in `package.json` exhibit flaky or inconsistent behavior caused by misconfigurations, version mismatches, or outdated packages. In such cases, the Dockerfile itself is correct, but the failure stems from issues within the external dependency definitions.

After filtering out these non-flaky failures, we were left with 974 build outputs originating from **798 (9.81%)** Dockerfiles out of 8,132 candidate projects, as some Dockerfiles exhibited multiple distinct flaky behaviors throughout our longitudinal analysis. This number would likely be even higher in real-world scenarios, as our candidate Dockerfiles were sourced from high-quality projects.

This percentage is significant when compared to the prevalence of flaky tests in practice. According to an empirical analysis on Flaky tests [14], 4.56% of all test failures across test executions at Google's continuous integration (CI) system, named TAP, were reported to be due to flaky tests during a 15-month window. Another study by Microsoft [19] reported that 4.6% of individual test cases monitored over a month were flaky. Our finding of 9.81% flaky Dockerfiles aligns with the prevalence of flaky tests in other large-scale systems, highlighting the importance of investigating and developing tools to address Dockerfile flakiness.

### E. RQ2: Taxonomy of Dockerfile Flakiness

Figure 1 illustrates our hierarchically structured taxonomy of Dockerfile flakiness. The left side shows the main categories of flakiness, while the right side lists the associated subcategories. The numbers within each box indicate the frequency of occurrences. For each subcategory, we indicate its nature—deterministic, non-deterministic, or mixed—where "mixed" refers to cases that exhibit both deterministic and



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

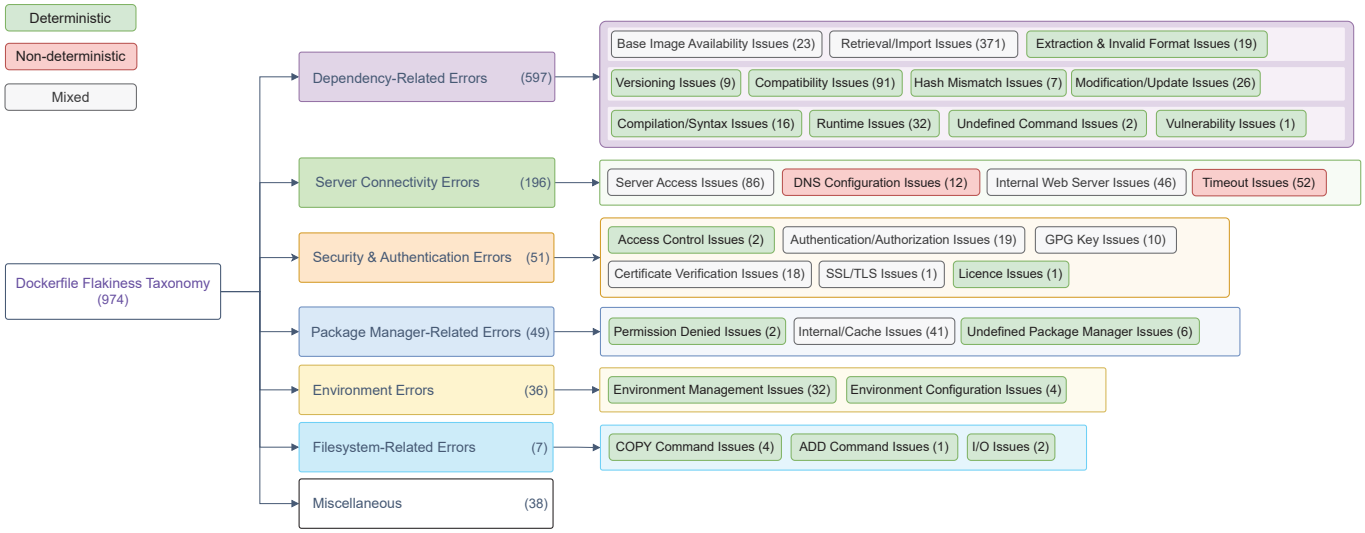



Fig. 1: Dockerfile Flakiness Taxonomy

non-deterministic behaviors. The rest of this section provides an overview of the primary categories of flakiness identified in our taxonomy. Detailed information, including examples for each category and sub-category, can be found in our replication package [20].

**Dependency-Related Errors (DEP).** This is the most prevalent category, accounting for 61.29% of all errors. It encompasses 11 subcategories of errors that occur during the retrieval, installation, or post-installation operations of dependencies specified in Dockerfiles. These three steps of errors are shown in the first, second, and third rows of dependency-related error subcategories in Figure 1. We define a *dependency* as a Base image or any external software package or library explicitly mentioned in the Dockerfile.

*Availability and Retrieval Errors:* These errors happen when previously available dependencies are not found or cannot be accessed (e.g. RETRIEVAL/IMPORT ISSUES) temporarily or permanently, or decompressed due to EXTRACTION & INVALID FORMAT ISSUES. As an example of BASE IMAGE AVAILABILITY ISSUES, one of the Docker projects we studied, `Mistserver` [21] , uses `FROM phusion/baseimage:master`, leading to a failure because this specific version of the image is currently unavailable.

*Installation Errors:* During the installation phase, flaky errors can occur due to different reasons including VERSIONING ISSUES or COMPATIBILITY ISSUES, where specific dependencies with altered versions or configurations may conflict or be incompatible with one another, causing the build process to fail. These errors can also involve HASH MISMATCH ISSUES and MODIFICATION/UPDATE ISSUES, where changes in dependencies lead to inconsistencies. For instance, consider a scenario where package `p1` requires package `p2` with a version greater than or equal to `v2` to be installed properly, but the existing `p2` version is older than `v2`.

*Post-Installation Errors:* After installation, flakiness can arise from the dependencies themselves, such as VULNERABILITY ISSUES, COMPILATION/SYNTAX ISSUES, or RUNTIME ISSUES. Additionally, improper installations can lead to UNDEFINED COMMAND ISSUES in the Dockerfile, resulting in build failures. As an example, we have observed flakiness in a Dockerfile using `Symphony`—a PHP framework—which is known to have vulnerability issues within its CodeExtension filters [22], and using those filters may cause flakiness.

**Server Connectivity Errors (CON).** This category is the second most prevalent among all the categories comprising 20.12% of the errors. These errors occur when there are issues while connecting to previously stable external servers. These errors are typically non-deterministic, except when servers are permanently out of service or entirely unreachable. SERVER ACCESS ISSUES arise when the Dockerfile cannot reach the target server due to invalid URLs or temporary server downtimes. TIMEOUT ISSUES occur when connections to servers take too long to establish or complete, often caused by overloaded servers or the massive size of transferred data. INTERNAL WEB SERVER ISSUES refer to errors within the accessed server, typically denoted with `HTTP` status codes `500s`. DNS CONFIGURATION ISSUES arise when the Dockerfile temporarily fails to resolve the server's domain name, resulting in failed connections.

**Security and Authentication Errors (SEC).** This category contains errors related to changes or deprecation of previous security protocols and authentication processes, making up 5.24% of the total errors. ACCESS CONTROL ISSUES arise when the Dockerfile does not have the necessary permissions to access required internal resources caused by the base image's internal changes. AUTHENTICATION/AUTHORIZATION ISSUES manifest when there are problems verifying the identity of the user or service, which can result from incorrect or expired credentials or misconfigured authentication/authorization services.



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

This subcategory is mixed, as such errors can be affected by transient factors like token expiration or permanent external service instability.

SSL/TLS ISSUES encompass a range of problems related to the secure transmission of data, including protocol mismatches and outdated cryptographic algorithms. GPG KEY ISSUES arise when there are problems with the cryptographic keys used to verify the integrity and authenticity of downloaded items, which can prevent the successful retrieval and installation of necessary dependencies. The nature of these errors is mixed, as they can result from invalid keys or temporarily unavailable key servers. Lastly, LICENCE ISSUES occur when the Dockerfile attempts to use software with new licensing restrictions.

**Package Manager-Related Errors (PMG).** Package manager-related errors constitute 5% of the flakiness instances and refer to the changes applied to the package manager configuration during the build process. The most common subcategory of this class is INTERNAL/CACHE ISSUES with 4.2% of total flakiness. These errors arise from inconsistency or unreliability in the package manager's internal system during its installation or utilization, resulting in failed operations. This subcategory is mixed, as deterministic failures can stem from corrupted caches, while non-deterministic failures may result from transient issues with the package manager's internal operations. As an example, the command: `RUN npm install --registry=r` where `r` is no longer a reliable registry for `npm` would cause an internal issue within the package manager. Another subcategory is PERMISSION-DENIED ISSUES, which occur when the Dockerfile does not have the necessary permissions to interact with the package manager to install or update packages. This sort of error can happen due to permission changes within the base images or other infrastructures within the Dockerfile. Lastly, UNDEFINED PACKAGE MANAGER ISSUES encompass errors caused by improper installation of package managers, resulting in a corrupted package manager within the system.

**Environment Errors (ENV).** ENVIRONMENT MANAGEMENT ISSUES and ENVIRONMENT CONFIGURATION ISSUES fall into this category, representing 3.7% of the errors corresponding to interactions with virtual environments. These environment errors often arise from changes made to the base images or other underlying infrastructures specified in the Dockerfile. Such changes enforce developers to strictly adhere to the new rules to minimize vulnerabilities and enhance the system's robustness. A detailed example of this type of error is illustrated in Listing 1 where an externally managed environment is required to alleviate the risk of disrupting the OS package management system.

**Filesystem-Related Errors (FS).** This category, representing the smallest portion of our study on Dockerfile flakiness, accounts for less than 1% of the total failures. These errors include challenges in handling file system operations within Dockerfiles such as COPY and ADD command errors, and I/O issues generally stemming from the base image internal file system updates.

**Miscellaneous.** During our analysis of Flaky Dockerfiles and their build outputs, we categorized 3.9% of the instances of flakiness as Miscellaneous. This group includes builds with highly complex errors or those executed in silent mode without informative execution logs, making it challenging to pinpoint the issues and classify them.

## IV. FLAKIDOCK

As demonstrated in Section III, Dockerfile flakiness presents various complex symptoms in the build output. Leveraging the ability of LLMs to solve programming tasks across different domains [23]–[33], and the effectiveness of retrieval-augmented generation (RAG) techniques [34]–[36], we propose FLAKIDOCK, an automated approach using LLMs to repair Dockerfile flakiness. Our insight is that by providing LLMs with demonstrations containing *static* (Dockerfile) and *dynamic* (build outputs) information, along with repair patches from similar examples, the model can resolve flakiness in new Dockerfiles. Figure 2 provides an overview of our approach.

### A. Demonstration Dataset Creation

We first need to create a demonstration dataset for our approach. To this end, we randomly sample 100 Dockerfiles from our dataset of 798 flaky Dockerfiles and manually provide repairs for them. To provide a robust demonstration of error categories in our repair dataset, we maintain a minimum of 30% coverage for categories that exhibit less than 5% distribution of Dockerfile flakiness, such as PMG, ENV, and FS categories. However, within each category, we randomly sample the Dockerfiles. If flakiness is not observed in a Dockerfile at the time of analysis, we randomly select another Dockerfile from our dataset for repair. We repair flaky Dockerfiles using static (Dockerfile), dynamic (build outputs), and extracted categorization information. If a repair solution is not apparent using this information, similar to SHIPWRIGHT [7], we perform a human inspection of the top five web pages from search engine results, based on querying the error keywords, to find potential solutions.

Repairing Dockerfile flakiness is a complex and time-consuming task because it involves several intricate steps. First, one must understand the execution steps of the Dockerfile and the overall workings of the project. Then, the specific problem must be pinpointed, and finally, the suggested repair should be verified. We verified the suggested repairs using the `Docker build` command with the `--no-cache` option. To confirm the reliability of the proposed repairs, we test them as many times as the longest consecutive failure streak observed in our build history. Additionally, if a Dockerfile has exhibited different types of flakiness throughout the flakiness extraction phase III-B, we only consider the types of flakiness that can be observed during the repair step. Some instances of flakiness may be difficult to reproduce or may have been resolved due to external dependency updates.

Table I shows the distribution of repairs per category. We spent approximately 115 person-hours generating 100



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

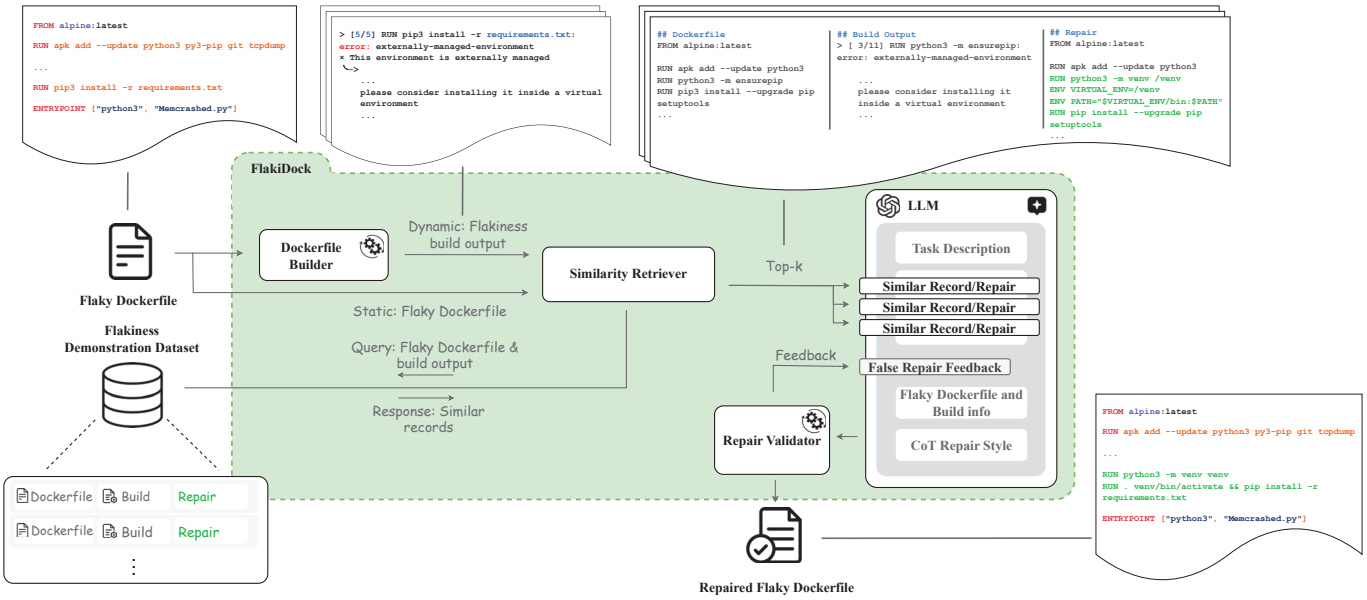



Fig. 2: Overview of our proposed approach: FLAKIDOCK

Dockerfile repairs. Notably, the low occurrence in the server connectivity category is due to the fact that most errors in this category are temporary server issues resolved by the time of analysis.

| Category | DEP | CON | SEC | PMG | ENV | FS | Total |
|---|---|---|---|---|---|---|---|
| **Repairs** | 63 | 6 | 9 | 8 | 10 | 4 | 100 |

TABLE I: Number of Repairs by Category

In our demonstration dataset, a Dockerfile is denoted with $d$, and every record is defined as tuple $(S_d, D_d, C_d, R_d, I_d)$ containing several elements of the Flaky Dockerfile. The $S_d$ component contains the static information, i.e., Dockerfile context. The identifier $D_d$ indicates the pre-processed build output of the flaky Dockerfile, which is referred to as the dynamic information. $C_d$ defines the category label for the current Dockerfile based on the taxonomy illustrated in Fig 1. Elements $R_d$ and $I_d$ denote the repair patch proposed for the Dockerfile and the number of iterations required for the repair to be tested to ensure its correctness, respectively. In case more than one repair is offered for a flaky Dockerfile, $R_d$ and $I_d$ form a list of values.

### B. Dockerfile Building

The first step of FLAKIDOCK is to build a given flaky Dockerfile to elicit the failing build output. The purpose of this building stage is two-fold. First, based on the build output extracted, we classify the Dockerfile as *non-flaky* if no failure is detected over $n$ iterations. Conversely, if a failure occurs, we identify it as flaky behavior and proceed to address it. $n$ is determined based on our demonstration dataset, ensuring it is at least greater than or equal to 90% of $I_d$ in our dataset, which corresponds to a minimum of two iterations. This approach is designed to encompass most flaky behaviors while optimizing time efficiency.

### C. Static and Dynamic Similarity Retrieval Phase

Few-shot learning has shown remarkable efficacy when applied to LLMs [37]. While randomly selected examples can enhance performance, recent studies [34], [35] indicate that choosing examples based on their similarity to the input context can lead to even more significant improvements.

In the domain of Dockerfiles, we argue that both static and dynamic information are required to resolve flakiness. To this end, upon completing the build phase, the captured information along with the original Dockerfile is used to retrieve similar examples from the flakiness demonstration dataset. Formally, the input given to the similarity retriever is a tuple $(S_q, D_q)$ where $S_q$ and $D_q$ represent the static (Dockerfile) and dynamic (build output) features respectively. Then, the retriever uses the pre-processing technique elaborated in Section III-B to extract error-related features from the build output. The combination of $S_q$ and $D_q$ is the input query $Q$ to the demonstration dataset. Given that our input encompasses both code segments and natural language descriptions, we utilize embedding-based search via `text-embedding-ada-002` embedding from OpenAI [38]. This model is pretrained on extensive datasets across various domains, can process up to 8,191 tokens, and outputs a vector of 1,536 dimensions. Using this transformer model, we compare $Q$ with the records in our demonstration dataset (only $S_e$ and $D_e$) by applying cosine similarity to retrieve the top-3 $(S_e, D_e)$ combinations that are closest in embedded space to the input query, along with the corresponding repair snippets $R_e$ for those retrieved Dockerfiles.



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

### D. Repair Generation

This step contains the prompt design for Dockerfile flakiness repair generation. As shown in Figure 2, the prompt comprises a natural language task description, the flaky Dockerfile along with its build output, and a Chain-of-Thoughts (CoT) explanation to guide the model through the repair process. The prompt is then augmented with demonstration examples retrieved from the similarity retriever. Each example $e$ consists of a triple: $(S_e, D_e, R_e)$ represents the Dockerfile, build output, and repair/repairs suggested for the flaky Dockerfile.

If previous attempts have generated incorrect repairs for the current flaky Dockerfile, a feedback message is included in the prompt as false demonstrations to help the model avoid similar mistakes. It encompasses the history of proposed repairs, each recorded as a false repair: $fr = (R_{fr}, D_{fr})$. Here, $R_{fr}$ element denotes the false repaired Dockerfile, and $D_{fr}$ shows the pre-processed build output corresponding to the false repair.

### E. Repair Validation

The validation phase serves as a heuristic approach to determine whether the repair suggested by the LLM is effective. The structure of this stage is elaborated in Algorithm 1. In the beginning, The repaired Dockerfile is built $n$ times, and build outputs are captured, similar to the Dockerfile builder module described in IV-B. If all build outputs are successful, the validator confirms the repair's correctness and finalizes it as a result. Otherwise, it identifies the most common error type observed from the feedback generated thus far. The feedback comparison relies on the similarity of build outputs, assessed using sentence transformation models.

Through a manual evaluation of our demonstration Dockerfiles, we found that after three incorrect repair attempts with the same error, LLMs tend to continue proposing flawed repairs due to hallucination or model deficiency in addressing that specific problem even with the augmented information provided. Therefore, we establish a threshold, denoted by $T$, set at a constant value of 3, to determine the stopping point for repair generation. If a specific error type appears $T$ times, we interpret it as an indication of the model's hallucination or inability to resolve the issue, resulting in the output `Unable to resolve!`. If no error occurs three or more times, new feedback—consisting of the false repair and its build output—is created, appended to the existing feedback list, and then incorporated into the LLM's prompt to generate a new Dockerfile flakiness repair.

**Algorithm 1** Repair Validation

**Input:**
$r_d \leftarrow$ Repaired Dockerfile,
$f_d \leftarrow$ Previous feedbacks
$it \leftarrow$ n
$T \leftarrow 3$ ▷ Threshold
**Output:** Repair / Feedback / "Unable to Resolve!"

$buildOutput \leftarrow$ getBuildResults($r_d$, $it$)
**if** allSuccessful($buildOutput$) **then**
  **Return:** $r_d$ ▷ Repair
**else**
  $failures \leftarrow countSimilarFailures(buildOutput, f_d)$
  **if** $failures \geq T$ **then**
    **Return:** `"Unable to resolve!"`
  **else**
    $f_d \leftarrow appendNewFeedback(r_d, buildOutput)$
    **Return:** $f_d$ ▷ Feedback
  **end if**
**end if**

## V. Evaluation

To assess the effectiveness of FlakiDock we address the following research question:

- **RQ3**: How effective is FlakiDock and how does it compare to state-of-the-art techniques?

### A. Implementation

FlakiDock is developed in Python. For our experiments, we use the GPT-4 model (`gpt-4-0613`) as the LLM, with the temperature parameter set to 0 to ensure deterministic and well-defined outputs. Chroma DB, which is an open-source embedding database [39], serves our need for vector similarity search. For embedding generation, we use `text-embedding-ada-002` from OpenAI [38]. The temporal analysis, including project checkouts and Dockerfile builds, is performed on an infrastructure consisting of 4 Intel(R) Xeon(R) CPU 2.50GHz machines with 62 GB RAM each. For repair, we use 8 AWS machines of type t2.2xlarge, each with 8 CPUs and 32 GB RAM.

### B. Experimental Setup

We initially identified 798 Dockerfiles exhibiting flaky behavior. Out of these, 100 Dockerfiles were reserved for *repair demonstration* purposes. This left us with 698 Dockerfiles showing signs of flakiness during our nine-month longitudinal study. However, for the evaluation phase, we focused only on those Dockerfiles that continued to exhibit flakiness at the time of evaluation. Consequently, we evaluated FlakiDock on 344 Dockerfiles, as some causes of flakiness had been resolved over time.

### C. Baselines

We compare FlakiDock with Parfum [5], Shipwright [7] and LLM-only prompting to repair Dockerfile flakiness. We chose Parfum as it is a recent work offering automated repairs for Dockerfile smells. Shipwright uses 13 repair-pattern rules to automatically repair Dockerfiles that fail to build. In contrast, other techniques, such as Hadolint [10] and Binnacle [3] focus on detecting error patterns or smells and require manual intervention for their operation. For GPT-4 Dockerfile Only, we invoke the GPT-4 model to generate repairs based solely on the Dockerfile content. Following that, we include build output to provide more context for the LLM to generate repairs.

We measure the effectiveness using the *repair accuracy* metric, which represents the percentage of genuine repairs produced. A proposed Dockerfile is considered a genuine repair if its build is successful across $n$ builds.



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Results for Dockerfile Flakiness Repair

| Tool and Strategy | DEP | CON | SEC | PMG | ENV | FS | Total |
|---|---|---|---|---|---|---|---|
| PARFUM | 0/280 (0%) | 1/9 (11.11%) | 0/16 (0%) | 1/16 (6.25%) | 0/22 (0%) | 0/1 (0%) | 2/344 (0.58%) |
| SHIPWRIGHT | 19/280 (6.79%) | 0/9 (0%) | 0/16 (0%) | 0/16 (0%) | 0/22 (0%) | 0/1 (0%) | 19/344 (5.52%) |
| GPT-4 Dockerfile | 13/280 (4.64%) | 2/9 (22.22%) | 1/16 (6.25%) | 4/16 (25%) | 1/22 (4.55%) | 0/1 (0%) | 21/344 (6.10%) |
| GPT-4 Dockerfile & build output | 105/280 (37.50%) | **4/9 (44.44%)** | 2/16 (12.50%) | 6/16 (37.50%) | 13/22 (59.09%) | 0/1 (0%) | 130/344 (37.79%) |
| **FLAKIDOCK (w/o feedback loop)** | 190/280 (67.86%) | 2/9 (22.22%) | 1/16 (6.25%) | 6/16 (37.50%) | **18/22 (81.82%)** | 0/1 (0%) | 217/344 (63.08%) |
| **FLAKIDOCK (w feedback loop)** | **216/280 (77.14%)** | **4/9 (44.44%)** | **6/16 (37.50%)** | **9/16 (56.25%)** | **18/22 (81.82%)** | 0/1 (0%) | **253/344 (73.55%)** |

### *D. Results*

Table II presents the repair accuracy of each method across flakiness categories. FLAKIDOCK (With Feedback Loop) achieves the highest success rate of 73.55%, demonstrating the effectiveness of iterative refinement. In comparison, PARFUM has the lowest accuracy at 0.58% and SHIPWRIGHT achieves 5.52%, both highlighting their limitations in addressing complex errors. GPT-4 Dockerfile Only achieves 6.10%, while GPT-4 Dockerfile & Build Output improves significantly to 37.79%, emphasizing the importance of incorporating build output context.

**Dependency-Related Errors (DEP).** The repair accuracy for DEP errors with PARFUM is 0%, as it relies on predefined rules that fail to address the complex and dynamic nature of dependencies, such as version mismatches or missing libraries. SHIPWRIGHT achieves 6.79%, which is notable as this is the only category it can handle, highlighting its limitations in addressing broader flakiness issues. GPT-4 Dockerfile Only achieves 4.64% accuracy, while GPT-4 Dockerfile & Build Output improves to 37.50% by leveraging build output context, enabling the LLM to better identify and resolve dependency issues. FLAKIDOCK, without a feedback loop, achieves 67.86% accuracy, significantly outperforming other methods. With the inclusion of feedback loop, FLAKIDOCK yields the highest accuracy of 77.14%, demonstrating the effectiveness of iterative refinement in resolving dependency-related errors.

**Server Connectivity Errors (CON).** PARFUM achieves an accuracy of 11.11% for addressing server connectivity errors. GPT-4 Dockerfile Only improves this to 22.22%, while the inclusion of build output further increases the accuracy to 44.44%, leveraging additional context for a more effective resolution. FLAKIDOCK, incorporating its feedback loop, achieves an equivalent accuracy of 44.44%, demonstrating performance comparable to the most effective LLM-based approach for this category.

**Security & Authentication Errors (SEC).** Security and authentication errors pose significant challenges due to their complexity, often requiring nuanced approaches beyond static rules. GPT-4 Dockerfile achieves a 6.25% success rate, reflecting limited effectiveness due to its reliance on general LLM capabilities without detailed context. Incorporating build output increases accuracy to 12.50%, highlighting the value of additional contextual information. Without a feedback loop, FLAKIDOCK also achieves 6.25%, underscoring the importance of iterative refinement. With the feedback loop, FLAKIDOCK attains the highest repair accuracy of 37.50%, demonstrating its ability to effectively address these intricate issues.

**Package Manager-Related Errors (PMG).** PARFUM achieves a modest success rate of 6.25%, reflecting its limited ability to handle this category. GPT-4 Dockerfile Only performs better, with a 25% success rate. This improvement is due to the LLM's general knowledge of package management, which enables it to address some common package-related issues based on Dockerfile content. Incorporating build output raises the accuracy to 37.50%, as the additional context enables better understanding of package manager-related problems. FLAKIDOCK, without a feedback loop, matches this 37.50% accuracy. However, when the feedback loop is incorporated, the highest accuracy achieved is 56.25

**Environment Errors (ENV).** GPT-4 Dockerfile achieves a 4.55% success rate, demonstrating limited effectiveness due to its reliance solely on Dockerfile content. Incorporating build output raises the success rate to 59.09%, as the additional context allows the LLM to better understand and resolve environmental issues. FLAKIDOCK outperforms all other methods, achieving an 81.82% success rate even without a feedback loop, demonstrating its strong performance in addressing environmental errors. However, incorporating a feedback loop does not provide additional improvement.

**Filesystem-Related Errors (FS).** For this flakiness category, all techniques, including FLAKIDOCK, fail to resolve any flakiness. The primary challenge lies in the inability of existing tools and LLMs to account for changes in the updated image's filesystem, such as a modified `WORKDIR`. Although feedback or similar examples could potentially aid in addressing these issues, our evaluation is limited to a single example, making it difficult to draw generalized conclusions about this category.

## VI. DISCUSSION

**(Non-)Deterministic Dockerfile Flakiness.** Dockerfile flakiness can manifest as deterministic or non-deterministic temporal failures, each requiring tailored repair approaches. Deterministic flakiness results in persistent and reproducible build errors caused by specific external changes, such as the deprecation of a dependency or removal of a required resource. For example, in Listing 3, a deterministic failure occurs due to the unavailability of the `npm.taobao.org` host, causing the



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

`wget` command to fail. FLAKIDOCK analyzes the static and dynamic build information, identifies the issue, and proposes a repair by substituting the unavailable URL with a valid one, as shown in the repaired Dockerfile. However, even after resolving a deterministic error, it is important to note that new issues may arise in the future due to the evolving nature of Dockerfile dependencies and environments.

**Build Output Summary**

```
wget: unable to resolve host address 'npm.taobao.org'
```

**Repaired Dockerfile**

```
- RUN wget https://npm.taobao.org/mirrors/node/v8.9.1/
      node-v8.9.1-linux-x64.tar.gz && \
+ RUN wget https://nodejs.org/dist/v8.9.1/node-v8.9.1-
      linux-x64.tar.gz && \
  tar -C /usr/local --strip-components 1 -xzf node-v8
       .9.1-linux-x64.tar.gz && \
  rm node-v8.9.1-linux-x64.tar.gz
```

Listing 3: Host Resolution Error (Deterministic)

In contrast, non-deterministic flakiness arises sporadically due to transient factors, such as server connectivity interruptions or temporary package manager caching inconsistencies. These failures lack a consistent pattern, complicating root-cause analysis and repair validation. For instance, Listing 4 demonstrates a timeout error caused by a transient network issue during a `yarn` install command. To address this, FLAKIDOCK adjusts the network timeout configuration in the Dockerfile to ensure stability, as shown in the repaired version. While FLAKIDOCK can propose an initial repair based on observed data, a single successful repair validation is insufficient. Instead, repairs for non-deterministic failures must be tested iteratively over time to ensure reliability and prevent reoccurrence. The repair validator in FLAKIDOCK facilitates this by running repeated builds, monitoring outcomes across multiple iterations, and confirming the robustness of the repair.

**Build Output Summary**

```
An unexpected error occurred: "https://registry.yarnpkg.
     com/@material-ui/icons/-/icons-4.11.3.tgz:
     ESOCKETTIMEDOUT".
```

**Repaired Dockerfile**

```
FROM node:16.14.0
...
COPY --from=deps /tmp/deps.json ./package.json
COPY yarn.lock ./
+ RUN yarn config set network-timeout 600000
RUN yarn install --production
```

Listing 4: Timeout Error (Non-Deterministic)

In FLAKIDOCK, deterministic flakiness is repaired with greater certainty due to its consistent root cause. Non-deterministic flakiness, however, requires iterative testing and extended validation to ensure repair effectiveness. FLAKIDOCK addresses both by offering immediate repairs for deterministic flakiness and incorporating a time-based validation process for non-deterministic cases, adjusting parameters such as $n$ for iterative validation.

**Threshold $n$ for Determining a Successful Repair.** The parameter $n$, representing the number of build iterations used to validate repairs, is crucial for ensuring repair accuracy in addressing Dockerfile flakiness. In our study, $n$ was set to two based on empirical analysis of the demonstration dataset, capturing over 90% of flaky behaviors. For temporal deterministic errors, this value is sufficient, as these errors are resolved once the root cause is addressed. Once a deterministic error has been repaired successfully in a single build, it is unlikely to reoccur, so additional iterations have little effect on repair accuracy.

Non-deterministic flakiness presents a more complex challenge. Since these errors can occur unpredictably, passing two rounds of validation may not suffice to reliably capture and resolve the flakiness, requiring a higher $n$ to ensure robustness. A potential enhancement to FLAKIDOCK would be the inclusion of a detector that identifies the type of Dockerfile flakiness (deterministic or non-deterministic). Based on this classification, and based on the retrieved similar records, the parameter $n$ could be adjusted accordingly. For deterministic flakiness, a lower $n$ is sufficient, while non-deterministic flakiness would require a higher $n$ to ensure a reliable repair. This would prevent premature validation of repairs for non-deterministic errors.

**Dynamic Nature of Dockerfile Flakiness.** Dockerfiles rely on evolving components such as base images, package managers, libraries, external URLs, and infrastructure. Changes in these dependencies such as deprecated libraries, broken URLs, or updated environment variables, can introduce flakiness, causing builds to fail. Conversely, flakiness can be resolved as dependencies stabilize without Dockerfile modifications, highlighting the temporal and dynamic nature of Dockerfile flakiness. Our nine-month study revealed the temporal nature of Dockerfile flakiness, where initially stable Dockerfiles became flaky due to dependency updates, and flaky ones stabilized as external issues were resolved. Static analysis tools fail to detect such failures, as they lack awareness of dynamic dependency changes (e.g., Listings 1 and 2). We propose a taxonomy to help developers systematically diagnose and address flakiness while preparing for future instability. This underscores the need for FLAKIDOCK, which dynamically adapts to evolving dependencies by providing context-aware repairs.

**Limitations of Existing Tools for Dockerfile Flakiness.** Existing tools for Dockerfile repair, such as SHIPWRIGHT and PARFUM, exhibit significant limitations when addressing the dynamic and multifaceted nature of Dockerfile flakiness. SHIPWRIGHT achieved only 19 repairs (5.52%) in our evaluation. However, this represents an upper bound, as a repair is deemed successful if SHIPWRIGHT generates an output Dockerfile using its predefined patterns, irrespective of whether it fully addresses the underlying flakiness. This tool relies on 13 repair patterns, heuristically designed by its authors through a clustering approach. These patterns must be maintained and updated to remain effective as new failure types emerge. With-



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

out regular updates, the effectiveness of the tool is likely to decline, given the dynamic and evolving nature of Dockerfile dependencies, which often introduce novel and unforeseen failure scenarios. SHIPWRIGHT was capable of handling only Dependency-Related Errors (DEP), likely because such errors often align with its predefined patterns, such as missing or incorrect dependencies. Similarly, PARFUM focuses on repairing Dockerfile smells but lacks mechanisms to address dynamic flakiness, achieving only a 0.58% repair accuracy. In contrast, our proposed tool, FLAKIDOCK, integrates both static and runtime contexts using retrieval-augmented generation and an iterative feedback loop, achieving a significantly higher repair accuracy of 73.55%. These results highlight the shortcomings of static, heuristic-driven tools and emphasize the need for dynamic, context-aware approaches to effectively handle the complexities of Dockerfile flakiness. However, FLAKIDOCK is not without limitations. It relies on LLMs, which may struggle with uncommon flakiness issues that fall outside their training scope. Its effectiveness depends on the diversity and quality of the demonstration dataset, limiting performance in addressing rare error categories. Additionally, the iterative repair process, while effective, can be computationally intensive and time-consuming.

## VII. THREATS TO VALIDITY

**Choice of Subjects.** The choice of Dockerfiles for our study can inherently introduce a bias, potentially influencing the results of our research. To address this, we employed a dataset from a prior study [7], encompassing 8132 Dockerfiles. However, the dataset includes Dockerfiles from repositories with ten or more stars, which one might find in popular GitHub repositories, addressing the generalizability of our findings to a broader range of Dockerfiles.

**Duration of the Study.** The duration of our study could potentially influence our findings, especially with respect to detecting Dockerfile flakiness and temporal failures. To address this, we conducted our study over a nine-month period. This allowed us to observe the Dockerfiles over a significant period, during which updates and changes are likely to occur, potentially affecting their behavior. This extended period provided us with sufficient data to identify non-deterministic behaviors and temporal inconsistencies effectively.

**Host Operating System.** The choice of operating system for Docker can affect the results. We used a stable version of Redhat 9 to ensure consistency, but this may limit the generalizability to other systems. Different Linux distributions, or versions, might show varying flakiness due to differences in package management and system libraries. Non-Linux hosts such as Windows or MacOS could exhibit different flakiness not captured in our study. By focusing on Redhat, we minimized environmental variability, isolating Dockerfile-specific flakiness. Future research could explore Dockerfile flakiness across diverse operating systems and versions.

**Repair Construction Bias.** As we generate the repair dataset, we might be biased in providing the repairs in a way that helps with other repairs. This bias could stem from the tendency to create repair patterns that are more easily generalizable, potentially overlooking unique or less common solutions. To alleviate this bias, we used repairs found from existing knowledge-sharing websites such as Stack Overflow, GitHub discussions, and the official Docker website.

## VIII. RELATED WORK

**Test Flakiness.** There is a wide array of techniques that have been proposed focusing on characterizing, detecting, and repairing flaky tests [14], [16], [17], [40], [41]. Continuous integration research such as [42] has a strong overlap with test flakiness literature, evaluating the prevalence and impacts of test flakiness in systems that involve test executions. Flaky tests are also a concern in user interface testing [43]. We refer to a recent survey for a more comprehensive discussion on flaky tests [44]. In contrast, we are the first to examine flakiness from the perspective of Dockerfiles.

**Dockerfile Analysis.** In Sections I and II, we discussed recent work on Dockerfile analysis [2]–[11]. FLAKIDOCK differentiates itself from existing Dockerfile smell detection tools [3], [8], [10], [11] and repair tools such as [5] by not only addressing static issues within Dockerfiles but also targeting the dynamic errors caused by the flakiness of Dockerfiles. SHIPWRIGHT [7] introduces a human-in-the-loop approach for repairing broken Dockerfiles. SHIPWRIGHT begins by clustering broken Dockerfiles to identify common failure patterns. This is achieved by embedding build log text into vector representations using a modified BERT language model, followed by clustering with the HDBSCAN [13] algorithm. Representative Dockerfiles from each cluster are then presented to a human supervisor, who searches for solutions to the identified failures. Discovered solutions are evaluated for applicability across the cluster, and successful fixes are formalized into a database of repair patterns. Each pattern comprises a regular expression to identify issues and a corresponding repair function detailing the fix. In contrast, FLAKIDOCK is fully automated, leveraging LLMs, retrieval-augmented techniques, and a feedback loop to refine repairs without requiring manual oversight.

**Learning-based Program Repair.** Learning-based program repair has been extensively studied in the literature [45]–[51]. Unlike these approaches, which involve training task-specific models, FLAKIDOCK uses a general-purpose LLM without the need for model training.

**LLM-based Program Repair.** There has been increasing focus on applying LLMs to program repair tasks. Early studies focused on using prompts and error messages to generate source code repair in a single interaction with the model [34], [52]. More recent techniques involve iterative approaches, querying the LLM multiple times and refining repairs based on feedback from previous attempts to repair source code [53]. In contrast, in this work, we leverage both static and dynamic information from Dockerfiles to provide the LLM, enhancing its ability to repair flakiness more effectively. By integrating



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

retrieval-augmented generation techniques, we further ensure that the LLM is equipped with relevant examples and contextual knowledge, leading to more accurate and reliable repairs.

## IX. Conclusion

We present the first comprehensive study on Dockerfile flakiness, identifying that 9.81% of Dockerfiles exhibit flaky behavior, significantly impacting the reliability of CI/CD pipelines. To address this issue, we introduce a novel taxonomy of Dockerfile flakiness and propose FlakiDock that leverages language models, retrieval-augmented generation, dynamic analysis, and an iterative feedback loop for automatic repair. Our evaluation shows that FlakiDock achieves a 73.55% repair accuracy, outperforming Parfum, Shipwright, and GPT-4-based prompting by 12,581%, 1,232%, and 95%, respectively. These results highlight the effectiveness of FlakiDock in addressing Dockerfile flakiness.

## X. Data Availability

We have made our dataset, model, comparison framework, and FlakiDock's implementation available [20] for the reproducibility of results.

## References


[1] "Docker," https://www.docker.com, 2024, accessed: 2024-01-28.

[2] G. Rosa, S. Scalabrino, G. Robles, and R. Oliveto, "Not all dockerfile smells are the same: An empirical evaluation of hadolint writing practices by experts," in *Proceedings of the 21st International Conference on Mining Software Repositories*, ser. MSR '24. Association for Computing Machinery, 2024, p. 231–241.

[3] J. Henkel, C. Bird, S. K. Lahiri, and T. Reps, "Learning from, understanding, and supporting devops artifacts for docker," in *42nd International Conference on Software Engineering*, ser. ICSE '20. Association for Computing Machinery, 2020, p. 38–49.

[4] Y. Zhou, W. Zhan, Z. Li, T. Han, T. Chen, and H. Gall, "Drive: Dockerfile rule mining and violation detection," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 2, 2023.

[5] T. Durieux, "Empirical study of the docker smells impact on the image size," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. Association for Computing Machinery, 2024.

[6] G. Rosa, S. Scalabrino, and R. Oliveto, "Fixing dockerfile smells: An empirical study," *arXiv preprint arXiv:2208.09097*, 2022.

[7] J. Henkel, D. Silva, L. Teixeira, M. d'Amorim, and T. Reps, "Shipwright: A human-in-the-loop system for dockerfile repair," in *43rd International Conference on Software Engineering*, ser. ICSE '21. IEEE Press, 2021, p. 1148–1160.

[8] Q.-C. Bui, M. Laukötter, and R. Scandariato, "Dockercleaner: Automatic repair of security smells in dockerfiles," in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2023, pp. 160–170.

[9] K. Eng and A. Hindle, "Revisiting dockerfiles in open source software over time," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, 2021, pp. 449–459.

[10] "Haskell dockerfile linter," https://github.com/hadolint/hadolint, 2023.

[11] "Linter and validator for dockerfile," https://github.com/replicatedhq/dockerfilelint, 2020.

[12] N. Reimers, "Sentence-bert: Sentence embeddings using siamese bert-networks," *arXiv preprint arXiv:1908.10084*, 2019.

[13] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Advances in Knowledge Discovery and Data Mining*, J. Pei, V. S. Tseng, L. Cao, H. Motoda, and G. Xu, Eds. Springer Berlin Heidelberg, 2013, pp. 160–172.

[14] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2014. Association for Computing Machinery, 2014, p. 643–653.

[15] "Python enhancement proposal (pep) 668," https://docs.docker.com/build/building/best-practices/, 2024, accessed: 2024-01-28.

[16] R. Wang, Y. Chen, and W. Lam, "ipflakies: a framework for detecting and fixing python order-dependent flaky tests," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, ser. ICSE '22. Association for Computing Machinery, 2022, p. 120–124.

[17] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "ifixflakies: a framework for automatically fixing order-dependent flaky tests," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019. Association for Computing Machinery, 2019, p. 545–555.

[18] "all-mpnet-base-v2: Sentence-transformers," https://huggingface.co/sentence-transformers/all-mpnet-base-v2, 2024, accessed: 2024-01-28.

[19] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2019. Association for Computing Machinery, 2019, p. 101–111.

[20] "Flakidock," https://github.com/flakidock/docker-flakiness-study, 2024.

[21] "Mistserver," https://github.com/R0GGER/mistserver/blob/0fe477e4fb35755ad0852d46c91ed42e5b18e990/Dockerfile, 2024, accessed: 2024-01-28.

[22] "Codeextension," https://github.com/symfony/symfony/security/advisories/GHSA-q847-2q57-wmr3, 2024, accessed: 2024-01-28.

[23] P. Denny, V. Kumar, and N. Giacaman, "Conversing with copilot: Exploring prompt engineering for solving cs1 problems using natural language," in *ACM Technical Symposium on Computer Science Education V. 1*. ACM, 2023, p. 1136–1142.

[24] S. I. Ross, F. Martinez, S. Houde, M. Muller, and J. D. Weisz, "The programmer's assistant: Conversational interaction with a large language model for software development," in *International Conference on Intelligent User Interfaces*. ACM, 2023, p. 491–514.

[25] Z. Xie, Y. Chen, C. Zhi, S. Deng, and J. Yin, "Chatunitest: a chatgpt-based automated unit test generation tool," *arXiv:2305.04764*, 2023.

[26] H. Tian, W. Lu, T. O. Li, X. Tang, S.-C. Cheung, J. Klein, and T. F. Bissyandé, "Is chatgpt the ultimate programming assistant–how far is it?" *arXiv:2304.11938*, 2023.

[27] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for 0.42 each using chatgpt," *arXiv:2304.00385*, 2023.

[28] S. Lertbanjongam, B. Chinthanet, T. Ishio, R. G. Kula, P. Leelaprute, B. Manaskasemsak, A. Rungsawang, and K. Matsumoto, "An empirical evaluation of competitive programming ai: A case study of alphacode," *arXiv:2208.08603*, 2022.

[29] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "An empirical cybersecurity evaluation of github copilot's code contributions," *arXiv e-prints*, pp. arXiv–2108, 2021.

[30] N. Nguyen and S. Nadi, "An empirical evaluation of GitHub Copilot's code suggestions," in *19th ACM International Conference on Mining Software Repositories (MSR)*, 2022, pp. 1–5.

[31] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM, 2022.

[32] A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, Z. Ming *et al.*, "Github copilot ai pair programmer: Asset or liability?" *arXiv:2206.15331*, 2022.

[33] S. Imai, "Is github copilot a substitute for human pair-programming? an empirical study," in *44th International Conference on Software Engineering (ICSE-Companion)*, 2022, pp. 319–321.

[34] N. Nashid, M. Sintaha, and A. Mesbah, "Retrieval-based prompt selection for code-related few-shot learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2450–2462.

[35] T. Ahmed, K. S. Pai, P. Devanbu, and E. Barr, "Automatic semantic augmentation of language model prompts (for code summarization)," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. Association for Computing Machinery, 2024.




Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.

[36] N. Nashid, T. Shabani, P. Alian, and A. Mesbah, "Contextual api completion for unseen repositories using llms," *arXiv preprint arXiv:2405.04600*, 2024.
[37] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *arXiv:2107.13586*, 2021.
[38] A. Neelakantan, T. Xu, R. Puri, A. Radford, J. M. Han, J. Tworek, Q. Yuan, N. Tezak, J. W. Kim, C. Hallacy *et al.*, "Text and code embeddings by contrastive pre-training," *arXiv preprint arXiv:2201.10005*, 2022.
[39] "Chroma db," https://www.trychroma.com/, 2024, accessed: 2024-01-28.
[40] Y. Chen, "Flakiness repair in the era of large language models," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, ser. ICSE-Companion '24. Association for Computing Machinery, 2024, p. 441–443.
[41] S. Fatima, H. Hemmati, and L. C. Briand, "Flakyfix: Using large language models for predicting flaky test fix categories and test code repair," *IEEE Trans. Softw. Eng.*, vol. 50, no. 12, p. 3146–3171, 2024.
[42] T. Durieux, C. Le Goues, M. Hilton, and R. Abreu, "Empirical study of restarted and flaky builds on travis ci," in *Proceedings of the 17th International Conference on Mining Software Repositories*, ser. MSR '20. Association for Computing Machinery, 2020, p. 254–264.
[43] A. Romano, Z. Song, S. Grandhi, W. Yang, and W. Wang, "An empirical analysis of ui-based flaky tests," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1585–1597.
[44] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–74, 2021.
[45] Z. Chen, S. J. Kommrusch, M. Tufano, L. Pouchet, D. Poshyvanyk, and M. Monperrus, "Sequencer: Sequence-to-sequence learning for end-to-end program repair," *IEEE Transactions on Software Engineering*, pp. 1–1, 2019.
[46] M. Sintaha, N. Noor, and A. Mesbah, "Dual slicing-based context for learning bug fixes," *Transactions on Software Engineering and Methodology (TOSEM)*, p. 27 pages, 2023.
[47] N. Nashid, M. Sintaha, and A. Mesbah, "Embedding context as code dependencies for neural program repair," in *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2023, pp. 95–106.
[48] M. Namavar, N. Nashid, and A. Mesbah, "A controlled experiment of different code representations for learning-based program repair," *Empirical Software Engineering*, vol. 27, no. 7, p. 190, 2022.
[49] M. Yasunaga and P. Liang, "Graph-based, self-supervised program repair from diagnostic feedback," in *37th International Conference on Machine Learning*. JMLR.org, 2020.
[50] B. Tang, B. Li, L. Bo, X. Wu, S. Cao, and X. Sun, "Grasp: Graph-to-sequence learning for automated program repair," in *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2021, pp. 819–828.
[51] Y. Ding, B. Ray, P. Devanbu, and V. J. Hellendoorn, "Patching as translation: The data and the metaphor," in *35th International Conference on Automated Software Engineering*. ACM, 2020, p. 275–286.
[52] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1482–1494.
[53] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for $0.42 each using chatgpt," *arXiv preprint arXiv:2304.00385*, 2023.



Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY ROORKEE. Downloaded on July 09,2026 at 19:05:55 UTC from IEEE Xplore. Restrictions apply.